

4장)_딥러닝

인공 신경망의 한계와 딥러닝 출현

퍼셉트론(perceptron)

- 인간의 논리 회로를 모방하여 이를 논리 회로 모델로 구성한 것

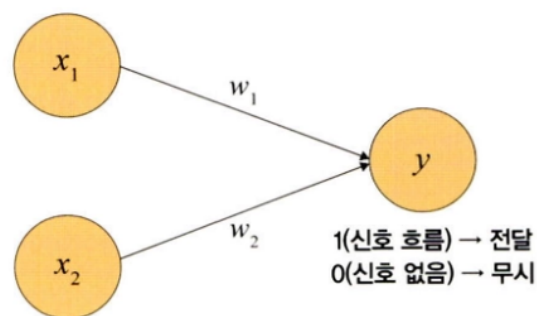
뉴런의 이해

- 가지돌기에 자극을 받게 되면, 신경세포체를 통해 축삭돌기에 화학물질이 전달되어, 전이가 발생하고
- 전이의 정도가 특정 임계값을 넘으면, 시냅스를 통해, 신호를 다음 뉴런에 전달하는 구조

인공신경망(artificial neural network, ANN)

- 뉴런의 작용을 0과 1로 이루어진 이진법 논리 회로 모델로 구성 → 이런 모델을 그물망처럼 연결하면, 인간의 뇌처럼 기계를 동작시킬 수 있을 것 (= 퍼셉트론)

♥ 그림 4-1 퍼셉트론 원리

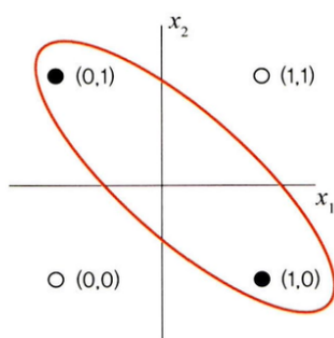


- $x_0, x_1, x_2, \dots, x_n$ 의 각각의 입력값을 받아서,
- $x_0 * w_0 + b, x_1 * w_1 + b, \dots, x_n * w_n + b$ 각 값을 가중치와 곱해서 (bias도 더함)
- $\sum_{i=1}^n (x_i * w_i + b)$ 이 특정 임계값을 넘으면 1, 그렇지 않으면 0과 같은 알고리즘으로 구성됨
- 예: if $\sum_{i=1}^n (x_i * w_i + b) > 0$ then 1, else 0
- 실제 결과와 오차가 생기면, 가중치를 업데이트함
 - 실제 결과와의 오차를 기반으로 가중치 업데이트를 할 수 있는 구조로 인해, 최초로 학습이 가능하게 됨

퍼셉트론의 이슈

1. 조건을 만족하는 선이 여러개 나와서, 최적의 선 구하기 어려움
 - [해결책] Adaline(아달라인) : 경사하강법을 도입하여 가중치를 업데이트시킴
2. 하나의 레이어를 가지고, 0과 1을 출력하는 퍼셉트론의 직선형태의 구분선으로는 XOR문제를 풀 수 없음

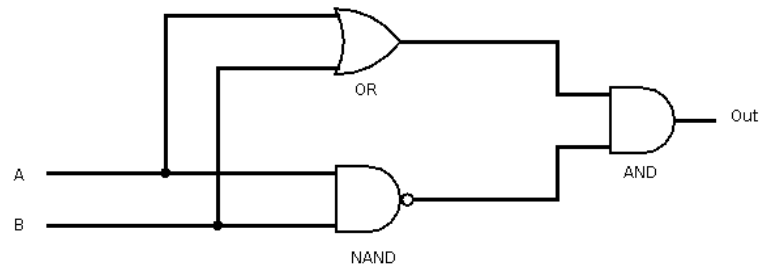
♥ 그림 4-4 XOR 게이트



♥ 표 4-3 XOR 게이트

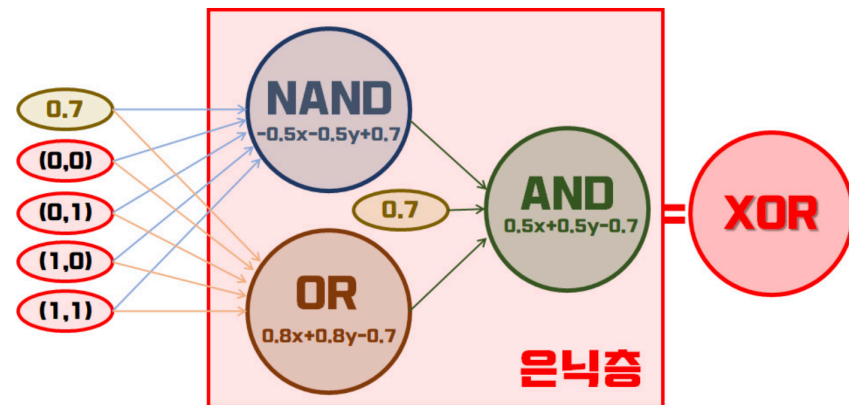
x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

- [해결책] NAND, OR, AND 게이트를 조합하면 XOR 게이트를 만들 수 있음



[출처] <http://sullystationtechnologies.com/npnxorgate.html>

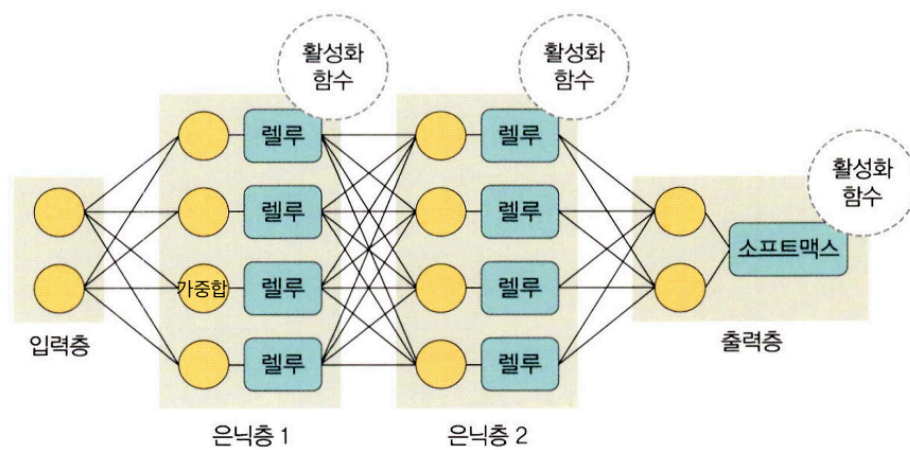
⇒ 즉, 여러 퍼셉트론 레이어(= 다중 퍼셉트론)를 두어, 은닉층을 만들어서 해결



XOR을 구현한 다중 퍼셉트론의 예시

⇒ 이때, 은닉층이 2개 이상인 신경망을 DNN(Deep Neural Network)라고 함

딥러닝 구조

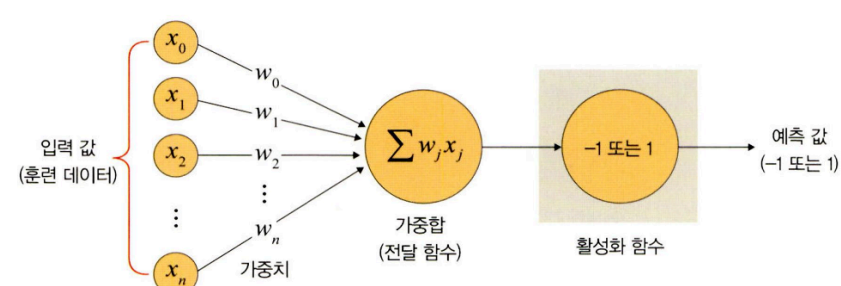


구분	구성요소	설명
층	입력층(input layer)	데이터를 받아들이는 층
	은닉층(hidden layer)	모든 입력 노드부터 입력 값을 받아 가중합을 계산하고, 이 값을 활성화 함수에 적용하여 출력층에 전달하는 층
	출력층(output layer)	신경망의 최종 결과값이 포함된 층

가중치

- 입력 값이 연산 결과에 미치는 영향력을 조절하는 요소
 - w_1 값이 0 혹은 0과 가까운 0.001이라면, x_1 이 아무리 큰 값이라도, $x_1 * w_1$ 값은 0이거나 0에 가까운 값이 됨

▼ 그림 4-6 가중치

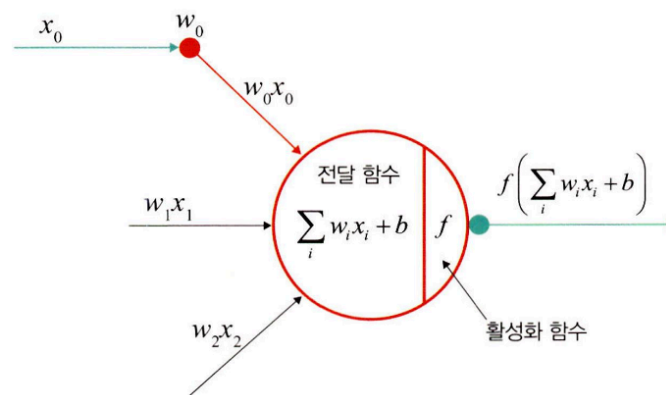


→ 이와 같이 연산 결과를 조정하는 역할을 하는 것이 가중치

가중합 또는 전달 함수

- 각 노드에서 들어오는 신호에 가중치를 곱해서 다음 노드로 전달되는데, 이 값들을 모두 더한 합계
 - $\sum_{i=1}^n (x_i * w_i + b)$
- 노드의 가중합이 계산되면, 이 가중합을 활성화 함수로 보내기 때문에 전달 함수(transfer function)이라고도 함

▼ 그림 4-7 전달 함수



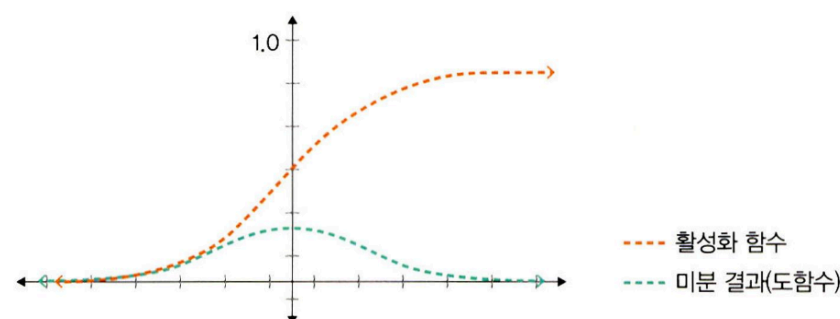
활성화 함수

- 가중합(전달 함수)에서 전달받은 값을 출력할 때 일정 기준에 따라 출력 값을 변화시키는 비선형 함수
- 세상 다양한 문제는 non-linear 문제가 더 많음 → 다층 퍼셉트론에서 activation function으로 non-linear function을 사용하여 non-linear 문제를 보다 잘 모사할 수 있도록 구성

1. Sigmoid 함수

- 선형 함수의 결과를 0~1 사이에서 비선형 형태로 변형해 줌
 - 수식: $\text{sigma}(x) = \frac{1}{1+e^{-x}}$
- sigmoid 함수는 머신러닝의 주요 모델 중 하나인 logistic regression 모델에서 많이 사용됨 → logistic regression은 이름에 regression이 들어 있지만 분류(Classification) 모델임
- **Logistic Regression**
 - Linear Classification(직선 모양)과 달리, **"S"**자 모양으로 곡선으로 계산하는 **Sigmoid function**을 사용해서 **분류**하는 모델
 - 정확히 0 또는 1을 예측하는 대신 확률(0과 1 사이의 값)을 생성해서 0과 1을 분류하는 예측 모델
- **Gradient Vanishing**가 발생하여, 현재 딥러닝 모델에서는 잘 사용되지 않음

▼ 그림 4-8 시그모이드 활성화 함수와 미분 결과

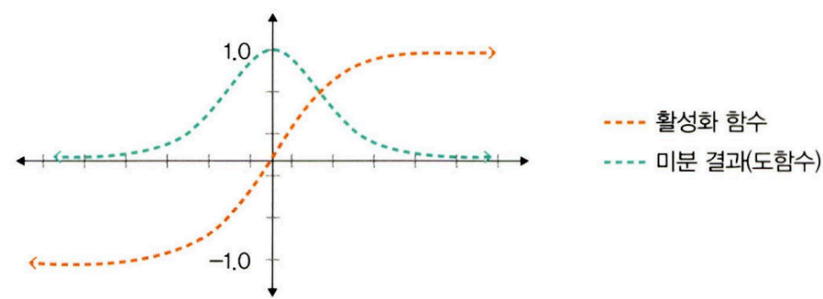


2. hyperbolic tangent 함수

- 선형 함수의 결과를 -1 ~ 1 사이에서 비선형 형태로 변형해 줌

- Sigmoid에서 결과값의 평균이 0이 아닌 양수로 편향된 분제를 해결하는데 사용했지만, Gradient Vanishing 문제는 여전히 발생

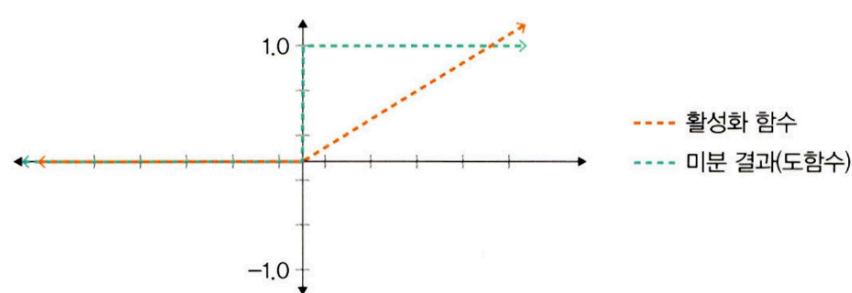
▼ 그림 4-9 하이퍼볼릭 탄젠트 활성화 함수와 미분 결과



3. ReLU 함수

- **Gradient Vanishing** 이슈를 해결하기 위해 고안된 **activation function**
- 수식: $f(x) = \max(0, x)$
 - 입력값 x 가 양수일 경우: $f(x) = x$
 - 입력값 x 가 0 이하일 경우: $f(x) = 0$
 - 출력 범위: $[0, \infty)$
- ReLU 함수의 미분은 $x \geq 0$ 일 때는 1, $x < 0$ 일 때는 0 이므로
 - 경사 하강법(gradient descent)에 영향을 주지 않아 학습 속도가 빠르고, Gradient vanishing 이슈를 어느 정도 보완할 수 있음
 - $x \geq 0$ 이면 미분이 1이므로 ,학습 속도가 sigmoid 보다 빠른 편임
 - sigmoid는 x 값이 커지면, y값이 1에 가까워지므로 기울기가 낮아져서 → 일정 값 이상이 되면 학습속도가 현저히 낮아짐
 - 일반적으로 은닉층에서 사용되며, hyperbolic tangent 함수 대비 학습 속도가 6배 빠름
- **ReLU 함수는 $x < 0$ 일 때는 미분값이 0이 됨 → 학습 능력이 감소함**
 - 가중치값으로 계산된 음수일 경우, 가중치 업데이트가 멈출 수 있음 (미분값이 0이기 때문)

▼ 그림 4-10 렐루 활성화 함수와 미분 결과



ReLU가 인기많은 이유

1. Mitigate vanishing gradient (기울기 소실 완화)

- 시그모이드(sigmoid)나 tanh 함수는 기울기(gradient)가 작아져서 역전파가 잘 안 되는 문제(기울기 소실)
- ReLU는 양수 구간에서 기울기 1로 일정하기 때문에, 학습이 훨씬 잘 됨

2. Simple computation (계산이 간단함)

- 비교 연산 하나로 끝남: $\max(0, x)$ → 빠르고 효율적. 딥러닝에서는 속도도 매우 중요!

3. Induce sparse activation (희소 활성화 유도)

- 입력값이 0보다 작으면 출력이 0 → 해당 뉴런은 **비활성화**됨
- 결과적으로 **전체 네트워크 중 일부 뉴런만 활성화**되기 때문에 → 계산이 줄어들고, **과적합 방지 효과**도 있음!

4. Simple but powerful non-linearity (단순하지만 강력한 비선형성)

- 비선형성(non-linearity)을 주기 때문에 **딥러닝 모델이 복잡한 함수도 학습 가능**
- 하지만 구조는 단순해서 **계산 효율성 + 표현력** 모두 만족

Sparse Activation이란

- ReLU가 어떻게 희소 활성화(sparse activation)를 유도하고, 왜 그것이 도움이 되는가
- **Sparse Activation (희소 활성화)란?**
 - 말 그대로 활성화된 뉴런이 적다는 뜻
 - (ex) 어떤 레이어에 100개의 뉴런이 있다고 가정 → 그런데 ReLU를 통과한 후 그 중 절반 이상이 **0**을 출력한다면 → 실제로 활성화(출력값이 0이 아닌 상태)된 뉴런은 소수 ⇒ 이게 바로 **희소 활성화**

- **왜 ReLU가 sparse activation을 만들까?**

$$\text{ReLU 함수의 정의: } \text{ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

- 즉, 입력값이 음수이거나 0이면 **출력 자체가 0**
 - 해당 뉴런은 **아예 아무 신호도 다음 층으로 보내지 않음** → 마치 "꺼진 뉴런"처럼 동작함 (**비활성화**됨)

- **희소 활성화의 장점은 뭘까?**

1. 계산 효율 향상

- 비활성화된 뉴런은 **곱셈/덧셈 연산이 필요 없음** → 전체 연산량이 줄고 속도 향상

2. 과적합 방지

- 모든 뉴런이 항상 활성화되면, 모델이 **너무 많은 정보를 학습**하게 되어 과적합될 수 있음
- 일부만 활성화되면 **정보가 분산**되고, 특정 뉴런이 너무 민감하게 반응하는 것을 막음
- 일종의 **Dropout처럼 regularization 효과**를 줌

3. 생물학적 뉴런 유사성

- 실제 뇌의 뉴런도 **일부만 활성화**되고 대부분은 **쉬고 있음** → ReLU의 희소성은 인간 뇌와도 유사한 특성

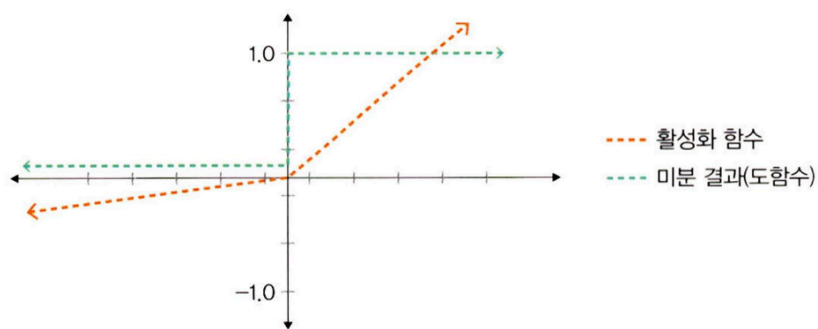
- **예시로 이해하기**

- 입력값 예시: $x = [-2, -1, 0, 1, 3, 5]$
- ReLU 적용 결과: $\text{ReLU}(x) = [0, 0, 0, 1, 3, 5]$
- 전체 6개 중 **3개는 출력 0** → 즉, 50%의 뉴런이 **비활성화**

4. Leaky ReLU 함수

- ReLU 함수의 문제를 해결하기 위해 나온 방안으로, 입력 값이 음수이면 0이 아닌 0.001처럼 매우 작은 수를 반환
 - 입력 값이 수렴하는 구간이 제거되어 ReLU 함수를 사용할 때 생기는 문제를 해결
- 수식 : $f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$

▼ 그림 4-11 리키 렐루 활성화 함수와 미분 결과



5. Softmax 함수

- 입력값을 0~1 사이에 출력되도록 정규화하여 출력 값들의 총합이 항상 1이 되도록 함
 - 보통 딥러닝에서 출력 노드의 활성화 함수로 많이 사용됨
- 수식 : $y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$
 - $\exp(x)$: 지수 함수
 - n : 출력층의 뉴런 개수, y_k : 그 중 k번째 출력
 - 소프트맥스 함수의 분자는 입력 신호 a_k 의 지수함수, 분모는 모든 입력 신호의 지수 함수 합으로 구성됨

손실 함수

- Gradient descent(경사 하강법)**은 학습률(learning rate)과 손실 함수(loss function)의 순간 기울기를 이용하여 가중치를 업데이트 하는 방법
 - 즉, 미분의 기울기를 이용하여 오차를 비교하고 최소화하는 방향으로 이동시키는 방법
 - 손실 함수(오차를 구하는 방법, 표본으로부터 추정된 예측값과 실제값의 차이)**는 학습을 통해 얻은 데이터의 추정치가 실제 데이터와 얼마나 차이가 나는지 평가하는 지표
 - 값이 클수록 많이 틀렸고, 0에 가까울 수록 완벽하게 추정할 수 있다는 의미

1. 평균 제곱 오차 (Mean Squared Error, MSE)

- 수식 : $MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$
 - N : 샘플 개수
 - y_i : 실제값 (정답 레이블)
 - $\hat{y}_i$: 모델의 예측값

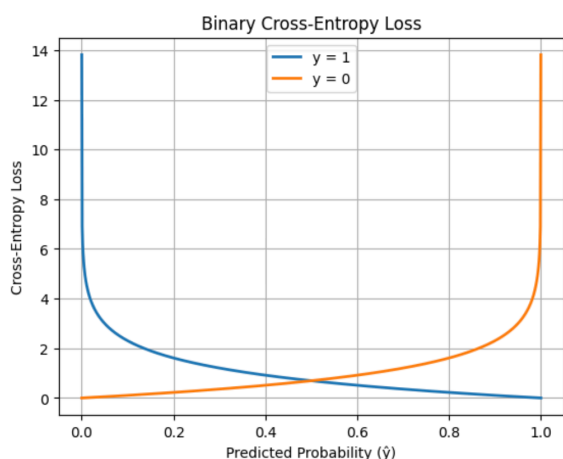
2. 크로스 엔트로피 오차 (Cross Entropy)

$$H(p, q) = - \sum_{x \in \text{classes}} p(x) \log q(x)$$

True probability distribution (one-shot) points to $p(x)$

Your model's predicted probability distribution points to $q(x)$

- 크로스 엔트로피 오차(CEE)는 **분류(classification) 문제에서 원-핫 인코딩(one hot encoding)**했을 때만 사용할 수 있는 오차 계산법
- 일반적으로 분류 문제에서는 데이터의 출력을 0과 1로 구분하기 위해 Sigmoid 함수를 사용하는데, 이 함수에 포함된 자연 상수 e 때문에 MSE를 적용하면 매끄럽지 못한 그래프(울퉁불퉁한 그래프)가 출력되어 크로스 엔트로피 손실 함수를 적용함
- 크로스 엔트로피 오차 손실 함수를 적용할 경우, Gradient descent 과정에서 학습이 local minimum에서 멈출 수 있기 때문에, 이를 방지하고자 자연 상수 e에 반대되는 자연 로그를 모델 출력 값에 취함



- 수식: $CE = - \sum_{i=1}^N y_i \log \hat{y}_i$
 - C : 클래스 개수
 - y_i : 원-핫 인코딩된 실제값 (해당 클래스이면 1, 아니면 0)
 - $\hat{y}_i$: 모델이 예측한 클래스 의 확률값

다중 분류에서의 Cross-Entropy Loss 공식

- $L = - \sum_{i=1}^C y_i \cdot \log(\hat{y}_i)$
 - C : 클래스의 개수
 - y_i : 실제 정답 레이블의 one-hot 인코딩 값 (정답 클래스는 1, 나머지는 0)
 - $\hat{y}_i$: softmax를 거친 예측 확률 (각 클래스에 대한 확률)
- 🖐 실제로는 정답 클래스 하나만 1이니까, 이걸 결국 이렇게 단순화돼요: $L = - \log(\hat{y}_{\text{정답 클래스}})$
 - 즉, 모델이 정답 클래스에 대해 예측한 확률만 보면 됩니다!

Softmax 함수가 왜 필요한가요?

- 다중 분류에서는 모델이 로짓(logit)이라는 스코어(정규화 전 값)를 출력해요.
 - Softmax는 이 로짓들을 확률로 바꿔주는 함수입니다: $\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$
 - z_i : 각 클래스에 대한 로짓
 - Softmax는 전체 확률의 합이 1이 되도록 만들어줍니다.

예제 상황

- 클래스 3개 (고양이, 개, 토끼)
- 정답: "고양이"
- 모델의 출력 (Softmax 결과): $\hat{y} = [0.7, 0.2, 0.1]$
- 정답 y (One-hot): $[1, 0, 0]$
- **Cross-Entropy Loss 계산:** $L = -(1 \cdot \log(0.7) + 0 + 0) = -\log(0.7) \approx 0.357$

• 만약 예측이 다음처럼 나왔다면?

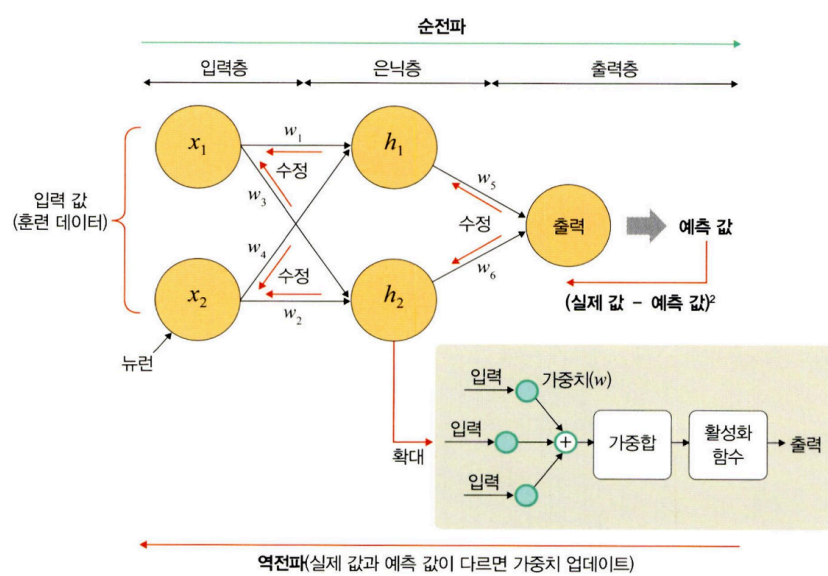
- $\hat{y} = [0.1, 0.2, 0.7]$ (정답인 고양이는 확률 0.1)
- $L = -\log(0.1) \approx 2.302$ (Loss가 매우 큼)

상황	손실값 (Loss)
정답 클래스에 높은 확률 부여	낮음 (좋음)
정답 클래스에 낮은 확률 부여	높음 (나쁨)

- Cross-Entropy는 정답 클래스에 얼마나 자신 있게 맞췄는지에 대해 보상 또는 벌점을 줍요.
- Softmax + CE는 분류 문제에서 거의 표준처럼 사용됩니다.

딥러닝 학습

▼ 그림 4-12 순전파와 역전파



1. 순전파(feedforward)

- 모든 뉴런이 이전 층의 뉴런에서 수신한 정보에 변환(가중합 및 활성화 함수)을 적용하여 다음 층(은닉층)의 뉴런으로 전송하는 방식

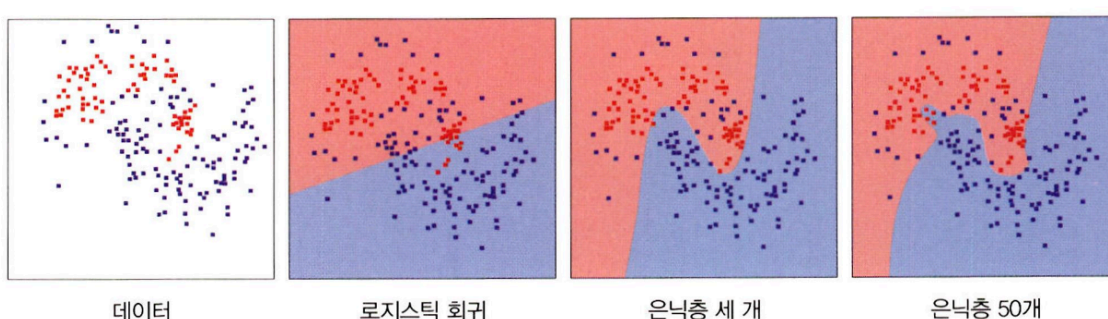
2. 역전파(backpropagation)

- 손실 함수로 네트워크의 예측 값과 실제 값의 차이(손실, 오차)를 추정 → 이때 손실 함수 비용이 0에 가깝도록 하기 위해 모델이 훈련을 반복하면서 가중치를 조정
 - 손실(오차)가 계산되면 그 정보는 역으로 전파(출력층→은닉층→입력층)되기 때문에 역전파
 - 출력층에서 시작된 손실 비용은 은닉층의 모든 뉴런으로 전파
 - but, 은닉층의 뉴런은 각 뉴런이 원래 출력에 기여한 상대적 기여도에 따라(가중치에 따라) 값이 달라짐
- ⇒ 예측 값과 실제 값 차이를 각 뉴런의 가중치로 미분한 후 기존 가중치 값에서 뺀 → 이 과정을 출력층→은닉층→입력층 순서로 모든 뉴런에 대해 진행하여 계산된 각 뉴런의 결과를 또 다시 순전파의 가중치 값으로 사용

딥러닝의 문제점과 해결 방안

- 활성화 함수가 적용된 은닉층의 개수가 많을수록 데이터 분류가 잘 되지만, 은닉층이 많을 수록 세 가지 문제점이 생김

▼ 그림 4-13 은닉층이 분류에 미치는 영향

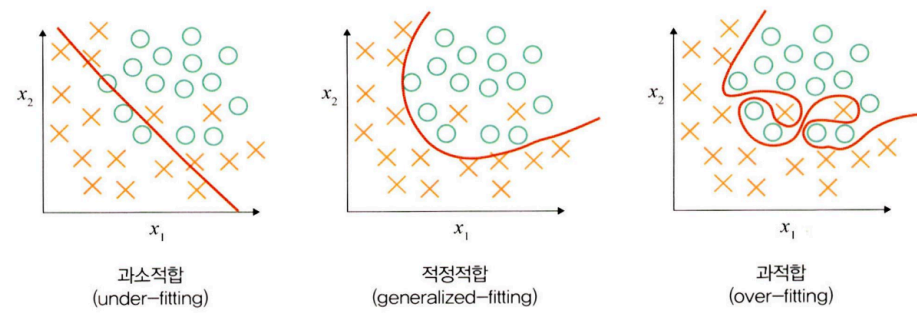


문제점

1. 과적합 문제(over-fitting) 발생

- 훈련 데이터에 대해 과하게 학습하여 실제 데이터에 대한 오차가 증가하는 현상

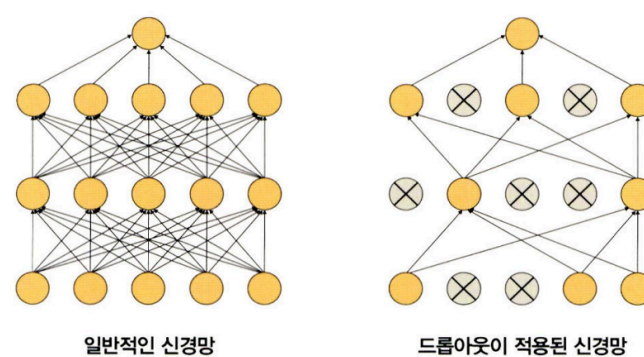
▼ 그림 4-14 과적합



- [해결방안] dropout

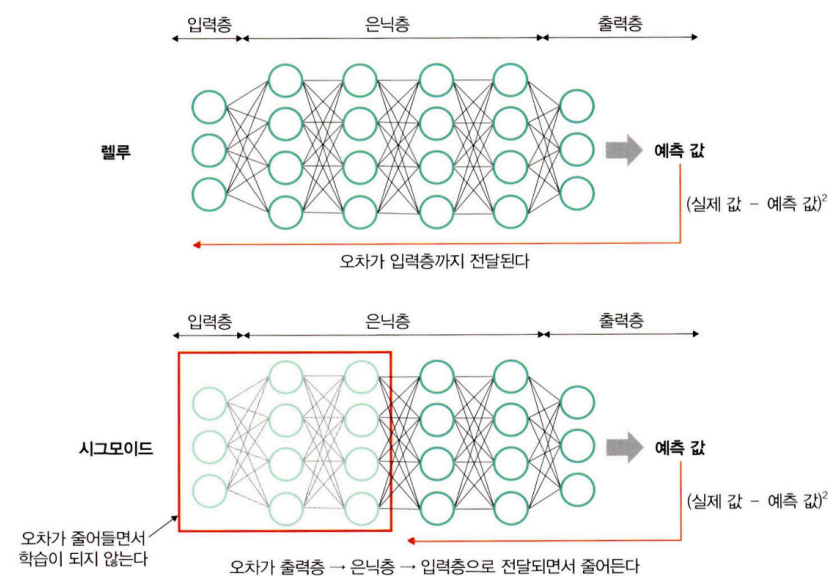
- 신경망 모델이 과적합되는 것을 피하기 위한 방법 → 학습 과정 중 임의로 일부 노드들을 학습에서 제외시킴

▼ 그림 4-15 일반적인 신경망과 드롭아웃이 적용된 신경망



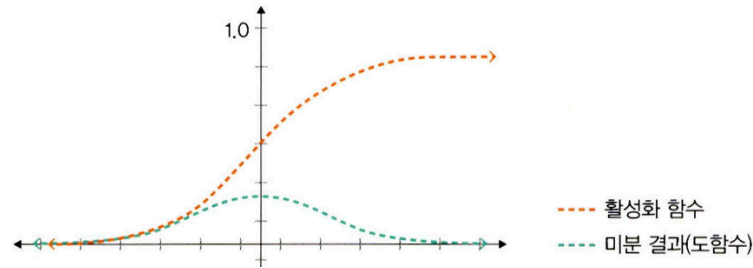
2. 기울기 소멸(Gradient Vanishing) 문제 발생

▼ 그림 4-16 기울기 소멸 문제



- **다층 퍼셉트론에 오차역전파 적용 시, 입력층으로 갈수록 기울기가 점차적으로 작아지는 현상** → 따라서, 입력층으로 갈수록 빼주는 값 자체가 작아져서(= 입력층으로 갈수록 그 가중치가 제대로 업데이트가 되지 않는 현상)
 - (↔ Gradient Exploding : 입력층에 갈수록 기울기가 비정상적으로 커져서 발산하는 현상)
- [이유] (ex) sigmoid 함수 : 결과값을 0과 1 사이의 값으로 조정하여 반환
 - 출력층의 마지막에 sigmoid 함수를 넣을 수도 있지만, 각각의 레이어마다 넣을 수 있음
 - sigmoid가 중간에 오차역전파를 할때도, 역시 연산에 포함되어있기 때문에 sigmoid 함수 자체도 미분이 됨
 - sigmoid 미분은 무조건 1보다 작음 → $x < 1$ 이면, x^{10} , x^{100} 과 같이 x 를 곱할 수록, 최종값은 0에 수렴하게 됨

▼ 그림 4-8 시그모이드 활성화 함수와 미분 결과

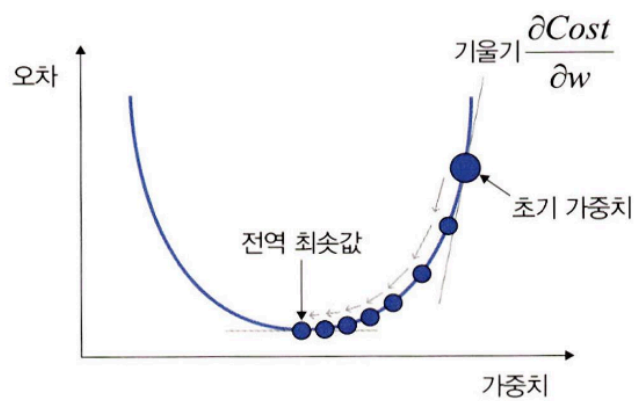


- activation function 미분 값이 1 보다 작을 경우, 레이어가 입력에 가까울 수록, 해당 값이 0에 가까워질 수 있음 → 결국 각 가중치 업데이트는 $w^{100} := w^{100} - \alpha \frac{\partial \text{Loss}}{\partial w^{100}}$ 이기 때문에, 가중치가 업데이트되기 어려움
- sigmoid 이외에도, **적용한 activation function의 미분 값이 1보다 작은 경우, Gradient Vanishing 현상이 발생 가능**

3. 성능이 나빠지는 문제 발생

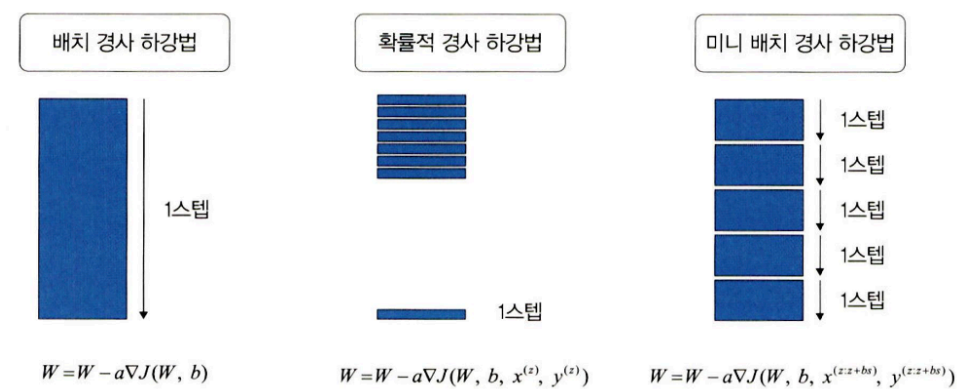
- 경사하강법은 손실 함수의 비용이 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동시키는 과정을 반복 → 이때 성능이 나빠지는 문제 발생

▼ 그림 4-17 경사 하강법



- [해결방안] : 확률적 경사 하강법, 미니 배치 경사 하강법

▼ 그림 4-18 경사 하강법의 유형



1. BGD(Batch Gradient Descent : 배치 경사 하강법)

- 전체 데이터셋에 대한 오류를 구한 후 기울기를 한 번만 계산하여 모델의 파라미터를 업데이트 → 전체 훈련 데이터셋에 대해 가중치를 편미분

2. SGD(Stochastic Gradient Descent : 확률적 경사 하강법) : 임의로 선택한 데이터에 대해 기울기를 계산하는 방법

- 적은 데이터를 사용하기 때문에 빠른 계산 가능
- 파라미터의 변경 폭이 불안정하고, 때로 배치 경사 하강법보다 정확도가 낮을 수 있지만 속도가 빠른 장점

3. Mini-Batch-Gradient Descent : 미니 배치 경사 하강법

- 전체 데이터셋을 미니 배치 여러 개로 나누고, 미니 배치 한 개마다 기울기를 구한 후 그것을 평균 기울기를 이용하여 모델을 업데이트해서 학습하는 방법
- 전체 데이터를 계산하는 것 보다 빠르고, 확률적 경사 하강법보다 안정적이고 속도도 빠름

옵티마이저(Optimizer)

- 학습 : 손실함수(loss function) 최소값을 찾아가는 과정 = 최적화(Optimization)
- 최적화(Optimization)을 수행하는 알고리즘 = 옵티마이저(Optimizer)

1. BGD (Batch Gradient Descent, 배치 경사 하강법)

- 전체 데이터셋에 대해 한번에 학습 (1 epoch = 1 iteration)
- 이 때의 Batch는 전체 데이터셋을 의미 → 전체 데이터셋을 기반으로 학습시키기 때문에 모델 파라미터 업데이트에 시간이 상대적으로 오래 걸림(한번 iteration을 도는 데에도 시간이 오래 걸림)
- 전체 데이터셋으로 학습시키기 때문에, 항상 입력이 일정하기 때문에, 수렴되는 정도가 일정함(안정적)
→ 수렴되는 정도가 안정되기 때문에, local minima 상태가 되면 빠져나오기 어려움

2. SGD (Stochastic Gradient Descent) : 확률적 경사 하강법

- 전체 학습 데이터 중 특정 크기만큼 임의로 선택해서 학습
 - 미니배치(Mini-Batch)
전체 데이터셋이 1000개가 있다고 할 때, 이를 200개의 Batch-Size로 분리하면, 각 200개의 서브 데이터셋을 Mini-batch라고 함
→ 1 epoch를 위해서는 200개씩 5번의 iteration이 필요함
- BGD의 단점을 보완하기 위해 고안 → 전체 데이터 중 랜덤하게 선택된 k개의 샘플 데이터를 이용하여 학습시키는 경사 하강법 (Mini-Batch-Gradient Descent)
 - 원래는 전체 데이터 중 1개의 샘플 데이터만 사용해서 학습시키는 경사하강법이 SGD임 → 최근에는 Mini-Batch-Gradient Descent를 SGD라고 부르는 경향이 있음
 - 전체 데이터가 500개 이고, batch size가 100이라면, 결국 1 epoch에는 5 iteration을 수행하게 되고, 전체 데이터셋에 대해 1000 epoch를 수행하고자 하면, 결국 5 X 1000 iteration의 학습 및 가중치 업데이트가 일어남

BGD / SGD / Mini-Batch GD 비교

항목	Batch GD (BGD)	Stochastic GD (SGD)	Mini-Batch GD
전체 데이터 수	N (예: 10,000개)	N (예: 10,000개)	N (예: 10,000개)
배치 사이즈	N (전체)	1	일반적으로 16 ~ 512 (예: 10)
미니배치 수 (= Iterations per Epoch)	1	N	N / 배치 사이즈 (예: 10,000 / 10 = 1000)
Epoch	전체 데이터 1회 사용 시 1 epoch	전체 데이터 1회 사용 시 1 epoch	전체 데이터 1회 사용 시 1 epoch
Iteration (총 업데이트 횟수)	Epoch 수 × 1	Epoch 수 × N	Epoch 수 × (N / 배치 사이즈) (예 : Epoch 수 X 1000)
업데이트 방식	에폭마다 1번	샘플마다 1번	미니배치마다 1번
특징	느리지만 정확한 방향	빠르지만 요동침	속도와 안정성의 밸런스

항목	Batch GD	SGD	Mini-Batch GD (배치 사이즈 = 10)
배치 사이즈	10,000	1	10
미니배치 수	1	10,000	1000
에폭 수	3	3	3
총 이터레이션 수 (총 업데이트 횟수)	3 × 1 = 3	3 × 10,000 = 30,000	3 × 1000 = 3000

1. BGD

- 전체 데이터를 한 번에 넣고 → 1번 업데이트
 - 학습은 느리지만 안정적
 - 실전에서 거의 안 씀 (데이터 클수록 느림)
-
- Batch size = # of training samples → 전체 데이터셋을 한꺼번에 사용해서 경사(gradient) 계산
 - 매번 일정하고 부드러운 직선 경로로 최적점에 가까워짐

- [이유] 전체 데이터 평균으로 계산하니까 노이즈가 없고 방향이 정확
- [단점] 큰 데이터일 경우 매번 계산이 너무 느리고 무거움
 - 항상 같은 path를 지나가기 때문에 local minima 벗어나기 어려움
- [장점] 안정적이고 수렴 속도도 빠름(단, 계산이 오래 걸릴 수 있음)

2. SGD

- 데이터 1개 볼 때마다 → **1번 업데이트**
 - **빠르지만 요동침**, 최솟값 근처에서도 계속 움직일 수 있음
 - 이론적으로는 좋지만 실전에서는 **Mini-Batch GD**가 더 안정적이라 대체됨
-
- **Batch size = 1 → 단 1개의 샘플만 사용해서 경사를 계산**
 - 매우 흔들리는 경로 -> 정확한 방향이 아니고 요동치면서 수렴
 - [이유] 한 샘플로 계산하니 편향되고 노이즈가 심함 -> 하지만 빠르게 움직임
 - 거의 random gradient라고 할 수 있음
 - 그러나 매번 다른 path 가기 때문에 가끔 batch gradient descent 만큼 혹은 더 잘 나오기도함
 - data도 학습하고 noise도 학습하기 때문에 더 generalized
 - Weight update per iteration
 - [단점] 흔들림이 너무 커서 안정적 수렴이 어렵고, 진동하면서 잘못된 방향으로 갈 수도 있음
 - [장점] 매우 빠르며, 때때로 local minimum을 탈출하기도 함(보장X)

3. Mini-Batch GD

- 데이터 100개씩 묶어서 → **1번 업데이트**
 - 전체 데이터가 10,000개면 100개짜리 묶음 100개로 나눔
 - **속도와 안정성 모두 잡아서** 가장 많이 쓰임
-
- **전체 데이터를 일정한 크기로 나눈 일부 데이터만 사용해 gradient 계산**
 - 살짝 지그재그지만 비교적 빠르고 안정적으로 수렴
 - [이유] 여러 배치의 평균값을 사용하므로 적당한 노이즈 -> 속도와 정확성의 균형
 - [단점] 여전히 조금의 노이즈 존재
 - [장점] 계산 효율이 좋고 GPU 병렬처리에 적합 -> 가장 널리 사용되는 방식

배치사이즈와 미니배치 수의 관계

- **배치 사이즈 = 미니배치 사이즈 → 같은 말**
 - **배치 사이즈(batch size)** : 한 번의 파라미터 업데이트에 사용되는 **데이터 샘플 수 (ex) 100**
= 도시락통의 **용량** (500ml)
 - **미니배치(mini-batch)** : 그 데이터 묶음 자체를 말하는 표현 (ex) $[x_1, x_2, \dots, x_{100}]$
= 그 도시락통에 실제 담긴 **음식 묶음**
-
- 예시
 - 전체 데이터: 10,000개
 - 배치 사이즈: 100

→ 그러면 데이터는 100개씩 나뉘어 → **100개의 미니배치가 생김**

→ 이때 **미니배치의 크기 = 배치 사이즈 = 100**

- 예시

- 전체 데이터 수 = **10,000개**, 배치 사이즈 = **1,000개**
- 배치 사이즈 : 1000
- 미니배치 수 (= Iterations per Epoch) : $10000/1000 = 10$ 개
- 1 Epoch당 파라미터 업데이트 횟수 : 10번 → 미니배치마다 1번씩 업데이트
- 전체 Epoch에서 파라미터 업데이트 횟수
 - 10,000개 데이터를 **1,000개씩 묶은 미니배치** 10개를 만들고,
 - 각 미니배치를 1번씩 학습 → **1 에폭 완료**
 - 이걸 에폭 수만큼 반복함 (예: 에폭 5라면 → 총 50번 업데이트)

```
[ 전체 데이터 10,000개 ]
├─ 미니배치 1 (1,000개) → 업데이트
├─ 미니배치 2 (1,000개) → 업데이트
├─ ...
└─ 미니배치 10 (1,000개) → 업데이트 → 1 에폭 완료

├─ 미니배치 1 (1,000개) → 업데이트
├─ 미니배치 2 (1,000개) → 업데이트
├─ ...
└─ 미니배치 10 (1,000개) → 업데이트 → 2 에폭 완료

├─ 미니배치 1 (1,000개) → 업데이트
├─ 미니배치 2 (1,000개) → 업데이트
├─ ...
└─ 미니배치 10 (1,000개) → 업데이트 → 3 에폭 완료

├─ 미니배치 1 (1,000개) → 업데이트
├─ 미니배치 2 (1,000개) → 업데이트
├─ ...
└─ 미니배치 10 (1,000개) → 업데이트 → 4 에폭 완료

├─ 미니배치 1 (1,000개) → 업데이트
├─ 미니배치 2 (1,000개) → 업데이트
├─ ...
└─ 미니배치 10 (1,000개) → 업데이트 → 5 에폭 완료
```

- 배치 사이즈가 커질 수록
 - 업데이트 횟수는 줄어듦 (100개씩 → 1,000개씩 처리하니까 이터레이션이 10번뿐)
 - 방향은 더 안정적

- 계산량은 커져서 메모리 부담 (한꺼번에 1,000개 처리해야 하니까)

미니배치 단위로 하나의 뉴런(neuron)에 들어가는 거야?

- Yes! 미니배치 단위의 데이터가 한 번에 들어가고, 각 데이터 샘플이 병렬로 여러 뉴런에 동시에 전달돼!

1. 미니배치란?

- 미니배치 = 한 번에 모델에 넣는 입력 샘플 묶음
- 예: `batch_size = 32` 이면 → 한 번에 32개의 샘플이 모델로 들어감

2. 뉴런은 어떻게 작동하냐면?

- 각 뉴런은 입력 데이터를 받아서 가중치 곱하고, 합치고, 활성화 함수 적용
- 이걸 한 샘플에 대해 반복하는 게 아니라, 미니배치 전체에 대해 벡터화/행렬 연산으로 동시에 처리함!

• 예시

- 미니배치 크기: 4
- 입력 특성 개수: 3 (즉, 각 샘플은 `[x1, x2, x3]` 형태)
- 뉴런 수: 2
- 그럼 입력은 이런 행렬이 됨

```
Input X (shape: 4 × 3)
[ x11 x12 x13 ]
[ x21 x22 x23 ]
[ x31 x32 x33 ]
[ x41 x42 x43 ]
```

- 가중치 W (뉴런 2개니까 3 x 2)

```
[ w11 w12 ]
[ w21 w22 ]
[ w31 w32 ]
```

- 결과

$Y = X @ W \rightarrow \text{shape: } 4 \times 2$

- 즉, 미니배치 전체(4개 샘플)가 한꺼번에 뉴런 2개에 병렬로 들어가고, 출력도 4개 샘플의 결과가 한 번에 나옴!

Learning Rate Schedules

방식	특징	대표 예시
고정 학습률	간단하지만 비효율적일 수 있음	기본 SGD
감쇠 스케줄	에폭에 따라 점점 줄어듦	Step, Cosine, Exponential

방식	특징	대표 예시
적응형 학습률	파라미터별로 자동 조절	Adam, RMSProp
워밍업	초반에 학습률을 천천히 증가	Transformer 학습 시 필수

- 딥러닝에서 학습률(Learning Rate)을 에폭(epoch)이 지남에 따라 **어떻게 조절할 수 있는지**
- 학습률은 모델이 얼마나 빠르게 가중치를 업데이트할지를 결정하는 **핵심 하이퍼파라미터**

1. Constant Learning Rate (고정 학습률)

어느 정도는 작동하긴 하지만, 대부분은 비효율적(suboptimal)

→ 처음엔 빠르게 수렴하지만, 나중엔 너무 커서 최적점 근처에서 흔들리거나 지나쳐버릴 수 있음.

- 학습률을 훈련 내내 변경하지 않고 일정하게 유지하는 방식
- 장점: 구현이 간단하고 직관적
- 단점: 학습이 잘 안 되거나 너무 빨리 발산할 수 있음

2. Decay Schedules (학습률 감소 스케줄)

이런 스케줄은 학습을 빠르게 시작하고, 안정적으로 마무리하고 싶을 때 좋음

- 시간이 지남에 따라 학습률을 점점 줄이는 전략
- 초기에는 빠르게 학습하고, 후반에는 안정적으로 최적점에 수렴하게 해줌
 - 초기에는 model이 untrained 상태이기 때문에 trained a lot 해야함
 - 후기에는 model을 많이 바꾸고 싶지 않고 fine tuning 하고 싶음
- 예시 방식들
 - Step Decay: 일정 에폭마다 학습률을 갑자기 줄임 (예: 10 에폭마다 절반으로 감소)
 - Exponential Decay: 학습률을 지수적으로 천천히 감소
 - Polynomial Decay: 다항식 기반으로 감소
 - Cosine Decay: 코사인 함수처럼 부드럽게 감소

3. Adaptive Learning Rates (적응형 학습률)

특히 고차원 파라미터 공간에서 더 정교한 학습을 가능하게 함

- 매 파라미터(또는 축)마다 학습률을 자동으로 조절하는 방식
- 대표적인 알고리즘: AdaGrad, RMSProp, Adam
- 장점
 - 파라미터마다 학습률을 다르게 조절 → 더 빠르고 효과적인 수렴
 - 손실이 잘 줄어들지 않는 방향엔 학습률을 키우고, 너무 민감한 방향엔 줄임

4. Learning Rate Warmup (워밍업 전략)

“처음엔 살살 달리다가, 모델이 익숙해지면 본격적으로 속도 낸다!”

- 학습 초반에는 학습률을 아주 작게 시작하고,
- 몇 에폭 동안 점차 증가시킨 후,
- 본격적인 학습률로 전환하는 방식
- 왜 필요한가요?
 - 처음부터 큰 학습률로 시작하면, 갑작스러운 gradient 변화 때문에 불안정할 수 있음

- 특히 Transformer, BERT 같은 대형 모델에서는 warmup 필수

How the weights are updated 딥러닝에서 가중치가 어떻게 업데이트 되는가

1. What if the loss function has a local minimum or saddle point?

= 손실 함수가 지역 최소점(local minimum)이나 안장점(saddle point)을 가지면 어떻게 될까?

- **Local Minimum (지역 최소점)**

- 그래프에서 더 이상 낮아지지 않는 지점이지만, 진짜 최저점(Global Minimum)은 아님
- 여기서 gradient(기울기)가 0이 되기 때문에 경사 하강법이 멈출 수 있음

- **Saddle Point (안장점)**

- 어떤 방향에서는 오르막, 다른 방향에서는 내리막인 지점
- 겉보기에 평평하거나 안정되어 보여도 사실은 불안정한 지형
- 마찬가지로 gradient가 0이 되기 때문에 경사 하강법이 멈출 위험이 있음.

- **이 두 경우 모두 기울기(gradient)가 0에 가까워지면, 학습이 더 이상 진행되지 않고 갇히게(stuck) 되는 문제 생김**

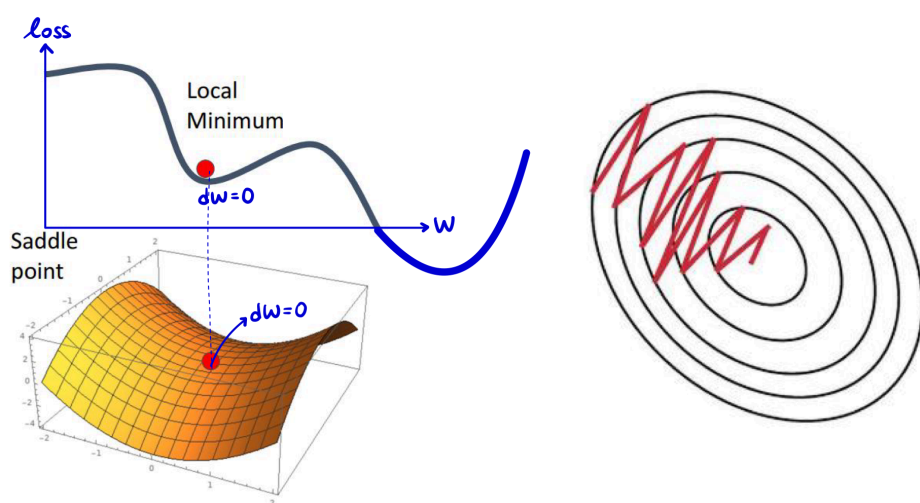
2. Zero Gradient → Gradient Descent gets stuck

- 경사 하강법은 기울기(gradient)가 있는 방향으로 이동하면서 손실(loss)을 줄이는 방식
- 그런데 local minimum이나 saddle point에서 기울기가 0이 되면
 - 가중치 업데이트가 멈추고 → **학습이 멈추는 현상**이 발생 가능

3. How can we accelerate training process with better convergence?

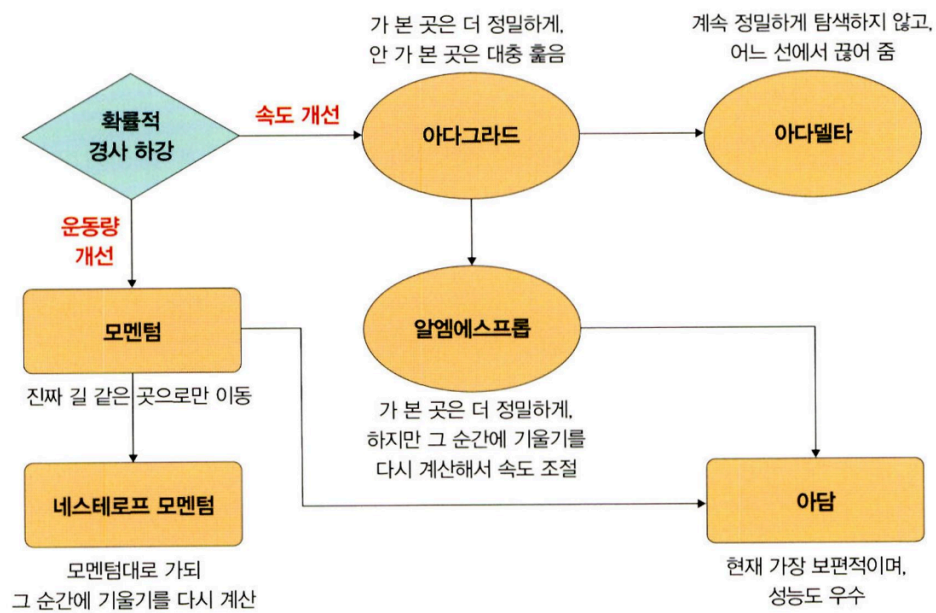
= 더 빠르고 안정적으로 수렴(convergence)하려면 어떻게 해야 할까?

- **Momentum**: 과거의 이동 방향을 고려해서 관성을 줌 → 흔들림 줄이고, saddle point에서 벗어나는데 도움
- **Adam, RMSProp**: 적응형 학습률로 방향을 더 똑똑하게 조절
- **Learning rate decay**: 처음엔 빠르게, 나중엔 천천히
- **초기값 잘 설정하기**: 나쁜 지점에서 시작하지 않게 조심



- **흔들리는 경로** → 이건 흔히 Stochastic Gradient Descent (SGD)에서 자주 보이는 모습
 - **노이즈 있는 gradient**를 따라 이동하다 보니 흔들리면서 이동하지만, 이런 랜덤한 움직임 덕분에 오히려 **local minimum**이나 **saddle point**를 빠져나올 수 있음

속도를 조정하는 방법



1. AdaGrad

- learning rate 값을 변경하는 알고리즘
 - SGD, Momentum, NAG 모두 모든 파라미터 업데이트에 동일한 learning rate 값 적용 → 학습률이 너무 작으면, 학습시간이 너무 길고, 학습률이 너무 크면, 발산해서 학습이 제대로 이루어지지 않음
- AdaGrad는 이를 learning rate decay 기법으로 개선하고자 함

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} \nabla E(w(i))$$

$$G(i) = G(i-1) + (\nabla E(w(i)))^2$$

- iteration 중, 많이 움직이는 경우(기울기가 컸던 경우), 해당 iteration 의 learning rate 는 그만큼 작아지므로, 학습률이 각 iteration 마다 다르게 적용됨 (각 iteration 별 맞춤형 learning rate 적용이 가능함)
- but, Adagrad는 t 가 증가함에 따라, G_t 가 점점 커져서 learning rate가 점점 소실되는 문제점이 있음

AdaGrad란?

- 각 파라미터(가중치)마다 고유한 학습률을 가지도록 자동 조절하는 알고리즘
 - 일반적인 SGD에서는 모든 파라미터에 동일한 학습률을 적용하지만,
 - AdaGrad는 gradient의 누적량에 따라 각 파라미터의 학습률을 다르게 조정**
 - 학습이 반복되면서 매번의 gradient(기울기)를 **제공해서 누적함으로 저장**
 - 파라미터를 업데이트할 때, 이 누적합의 **제곱근으로 나눠서 학습률을 줄이는** 방식

```

grad_squared = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    grad_squared += dw * dw
    w -= learning_rate * dw / (grad_squared.sqrt() + 1e-7)
  
```

- 수식

$$g_t = \nabla f(w_t)$$

$$G_t = \sum_{i=1}^t g_i^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot g_t$$

- g_t : 현재 gradient
- G_t : gradient 제공 누적값
- η : 기본 학습률
- ϵ : 나누기 0 방지용 작은 수 (예: $1e-7$)

• 장점 (Benefits)

- 자주 업데이트되는 파라미터는 누적된 gradient가 커서 $\rightarrow$ 학습률이 점점 작아짐 $\Rightarrow$ 학습이 과도하게 이뤄지는 것을 방지
- 거의 업데이트되지 않는 파라미터는 $\rightarrow$ 학습률이 상대적으로 큼 $\Rightarrow$ 학습이 따라잡을 수 있도록 도움
- 즉, 파라미터 간 학습 불균형을 자동으로 보정해주는 역할!

• 단점 (Limitations)

- gradient 제공을 계속 누적하다 보면 $\rightarrow$ 분모가 점점 커져서 $\rightarrow$ 학습률이 거의 0에 가까워질 수 있음 $\Rightarrow$ 학습이 멈추는 문제 발생
- 이 문제를 보완하기 위해 **RMSProp, Adam** 같은 알고리즘이 등장

• 예시 설정

- 초기값:
 - $\text{learning_rate} = 1.0$
 - $\epsilon = 1e-7$ (매우작아서 무시해도 무방)
 - $w = 10$
 - $\text{grad_squared} = 0$
- 각 타임스텝별 gradient dw :
 - $t = 1 : dw = 2$
 - $t = 2 : dw = 2$
 - $t = 3 : dw = 2$
 - $t = 4 : dw = 2$ (즉, gradient가 매번 동일하다고 가정)

• $t = 1$

$$\begin{aligned}\text{grad_squared} &= 0 + 2^2 = 4 \\ \text{w update} &= 10 - \frac{1.0 \times 2}{\sqrt{4}} = 10 - \frac{2}{2} = 9\end{aligned}$$

• $t = 2$

$$\begin{aligned}\text{grad_squared} &= 4 + 2^2 = 8 \\ \text{w update} &= 9 - \frac{1.0 \times 2}{\sqrt{8}} \approx 9 - \frac{2}{2.828} \approx 9 - 0.707 \approx 8.293\end{aligned}$$

- **t = 3**

$$\text{grad_squared} = 8 + 2^2 = 12$$

$$\text{w update} = 8.293 - \frac{2}{\sqrt{12}} \approx 8.293 - \frac{2}{3.464} \approx 8.293 - 0.577 \approx 7.716$$

- **t = 4**

$$\text{grad_squared} = 12 + 2^2 = 16$$

$$\text{w update} = 7.716 - \frac{2}{\sqrt{16}} = 7.716 - \frac{2}{4} = 7.716 - 0.5 = 7.216$$

t	dw	grad_squared	sqrt	update amount	w 값
1	2	4	2.000	1.000	9.000
2	2	8	2.828	0.707	8.293
3	2	12	3.464	0.577	7.716
4	2	16	4.000	0.500	7.216

- **grad_squared 값이 누적되면서 분모가 커짐** → 그래서 **업데이트 폭이 점점 작아지고 있음**
- 학습률이 자동으로 **점점 줄어드는 효과**가 생김 ⇒ **점점 천천히 수렴**하면서 안정성을 얻지만, 너무 작아지면 **학습이 멈출 위험도 있음**

2. Adadelta

- 아다그라드에서 G 값이 커짐에 따라 학습이 멈추는 문제를 해결하기 위해 등장한 방법
- 아다그라드의 수식에서 학습률을 D 함수(가중치의 변화량 크기를 누적인 값)로 변환했기 때문에 학습률에 대한 하이퍼파라미터가 필요 없음

$$w(i+1) = w(i) - \frac{\sqrt{D(i-1)+\epsilon}}{\sqrt{G(i)+\epsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

$$D(i) = \gamma D(i-1) + (1-\gamma)(\Delta(w(i)))^2$$

3. RMSProp

- Adagrad 에서 발생한 t 가 증가함에 따라 learning rate가 점점 소실되는 문제를 해결
- G 를 구할 때 합을 이용하지 않고, Exponential Weighted Moving Average(지수 가중 평균)를 이용

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i)+\epsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

- 시간이 흐름에 따라, 오래된 데이터의 영향이 지수적으로 감쇠함

AdaGrad VS RMSProp

- **RMSProp (Root Mean Square Propagation)**
 - “누적값에 가중치를 두어 오래된 gradient는 서서히 잊자!”
 - **AdaGrad는 모든 과거 gradient를 동일하게 반영하지만**
 - **RMSProp은 지수 감쇠(Exponential Decay)를 적용해서**

→ 최근 **gradient**는 크게 반영하고

→ 오래된 **gradient**는 점점 잊어버림

```
grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dw * dw
w -= learning_rate * dw / (grad_squared.sqrt() + 1e-7)
```

- **decay_rate** 는 보통 **0.9 ~ 0.99**
- **grad_squared** 는 이제 "지수 이동 평균" 형태의 누적값이 됨
- 이렇게 바뀌면서 좋은 점
 - **grad_squared** 가 무한히 커지지 않음 → 학습률이 너무 작아지는 문제 해결 → 지속적인 학습 가능!

항목	AdaGrad	RMSProp
누적 방식	모든 gradient를 누적	지수적으로 가중치 주며 누적
학습률 변화	시간이 지날수록 급격히 감소	완만하게 조절
장점	희소 데이터에 강함	안정적인 학습 유지
단점	학습률이 너무 빨리 0 근처로 감	오래된 정보는 잊지 못함

- **AdaGrad**: "모든 기억을 무조건 다 저장하는 학생" → 결국 머리 터짐
- **RMSProp**: "최근 내용은 집중하고, 예전 내용은 자연히 흐릿해지는 똑똑한 학생"

운동량을 조정하는 방법

1. Momentum

- SGD에 관성이 추가된 Optimizer → 관성 적용을 위해, velocity term(v)이 추가됨

$$\begin{aligned}w(i+1) &= w(i) - v(i) \\v(i) &= \gamma v(i-1) + \eta \nabla E(w(i))\end{aligned}$$

- 매 단계마다 gradient가 동일한 부호를 가지면, v 의 절대값은 계속 증가하게 됨 → 즉, 같은 방향으로 이동하면 할수록 가속도가 붙게 됨
- 따라서, local minima에서 gradient가 0에 수렴해도 이전 단계에서 축적된 v 값에 의해서 local minima 등을 탈출할 수도 있음

SGD(확률적 경사 하강법)에 Momentum(모멘텀)을 추가했을 때

- **SGD + Momentum (모멘텀 기법 추가)**
 - "이전 방향을 조금 더 기억하면서 나아가자!"
 - 즉, 단순히 현재 gradient만 따라가는 게 아니라, 이전에 쌓인 방향(속도, velocity)도 고려해서 더 빠르고 부드럽게 이동하게 만듦

- 수식

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

- v_t : 누적된 속도 (이전 gradient 정보 포함)
- ρ : 모멘텀 계수 (보통 0.9 또는 0.99 사용) → **마찰처럼 작동**, 값이 클수록 과거 방향을 더 오래 유지

```

v = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    v = rho * v + dw
    w -= learning_rate * v

```

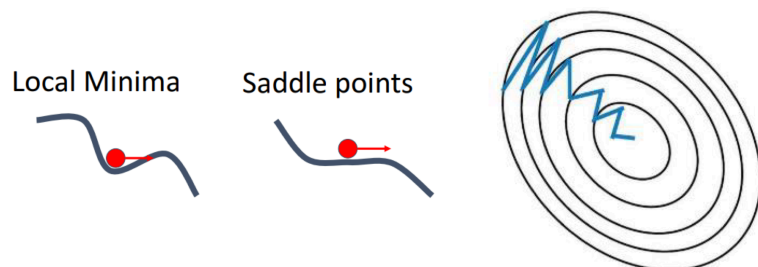
• 왜 모멘텀이 효과적일까?

1. 비슷한 방향 gradient → 더 빨라짐

- 반복해서 같은 방향의 gradient가 나오면, 속도가 누적되면서 더 크게 이동 → 빠른 수렴

2. 진동 억제

- gradient가 **엇갈리게 왔다 갔다 할 때**, 그 변화들이 **서로 상쇄되어** 진동이 줄어듦 → 더 부드럽고 안정적인 경로로 이동



• Local Minima (지역 최소점)

- 그냥 SGD는 여기에 빠져서 멈추기 쉬움 → Momentum은 과거 방향의 힘을 이용해 탈출 가능성 증가

• Saddle Point (안장점)

- gradient가 0에 가까워서 SGD는 멈춤 → 하지만 momentum은 이전 속도가 남아있어 지나칠 수 있음

• 곡면에서의 경로

- SGD만 쓰면 지그재그로 진동 → Momentum을 쓰면 매끄럽고 빠르게 최저점으로 이동

2. NAG

- momentum 방식은 이동을 중지해야하는 지점에 도달해도 momentum에 의해 해당 지점을 지나칠 수 있다는 문제를 가짐 → 이런 문제점을 해결한 것이 NAG
- momentum 계산 시 momentum에 의해 발생하는 변화를 미리 보고 momentum을 결정

$$\begin{aligned}
 w(i+1) &= w(i) - v(i) \\
 v(i) &= \gamma v(i-1) + \eta \nabla E(w(i) - \gamma v(i-1))
 \end{aligned}$$

- 현재 W_t 의 위치에서 W_{t+1} 의 gradient를 계산하고자 함 → 현재가 아닌 다음 단계의 기울기를 가지고 가중치를 업데이트 하려고 한 것!

속도와 운동량에 대한 혼용 방법

- Adam
 - 가장 많이 사용하는 optimizer → Momentum과 RMSProp를 결합한 느낌

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} v(i)$$
$$G(i) = \gamma_2 G(i-1) + (1 - \gamma_2)(\nabla E(w(i)))^2$$
$$v(i) = \gamma_1 v(i-1) + \eta \nabla E(w(i))$$

- momentum의 장점인 이전 gradient의 경향을 적용
- RMSProp의 장점인 learning rate가 점점 소실되는 문제 해결 → 이전 learning rate의 경향을 적용

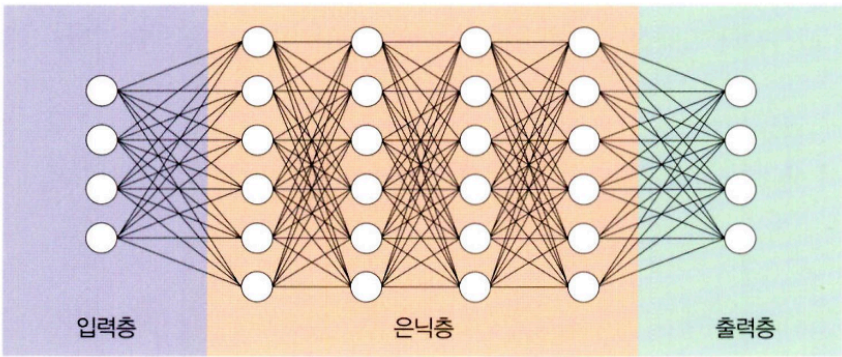
딥러닝을 사용할 때 이점

- 특성 추출 : 데이터 특성을 잘 잡아내고자 은닉층을 깊게 쌓는 방식으로 파라미터를 늘린 모델 구조
→ 특성 추출 과정을 알고리즘에 통합시킴
- 빅데이터의 효율적 활용 : 특성 추출을 알고리즘에 통합시키는 것이 가능한 이유는 빅데이터 덕분
→ 딥러닝 학습을 이용한 특성 추출은 사례가 많을 수록 성능이 향상됨

딥러닝 알고리즘

심층 신경망 (DNN)

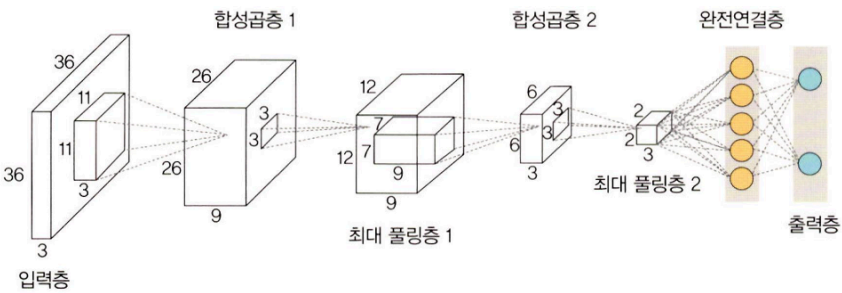
▼ 그림 4-24 심층 신경망



- 입력층과 출력층 사이에 다수의 은닉층을 포함하는 인공 신경망
 - 다수의 은닉층을 추가했기 때문에 별도의 트릭 없이 비선형 분류가 가능
 - 학습을 위한 연산량이 많고 Gradient Vanishing 문제가 발생할 수 있음
 - [해결방안] dropout, ReLU, 배치 정규화 등을 적용해야함

합성곱 신경망 (CNN)

▼ 그림 4-25 합성곱 신경망

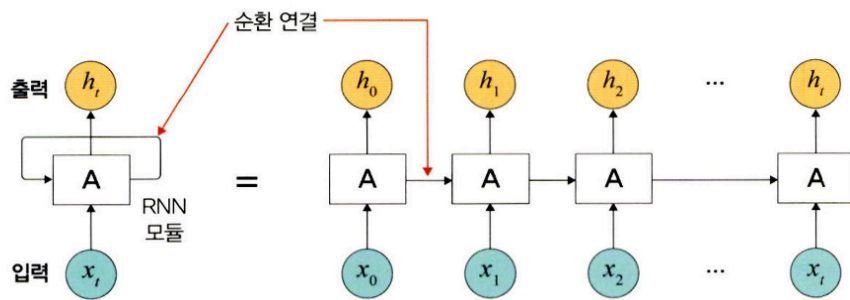


- 합성곱층(convolutional layer)과 풀링층(pooling layer)을 포함하는 이미지 처리 성능이 좋은 인공 신경망 알고리즘

- 영상 및 사진이 포함된 이미지 데이터에서 객체를 탐색하거나 객체 위치를 찾아내는 데 유용

순환 신경망 (RNN)

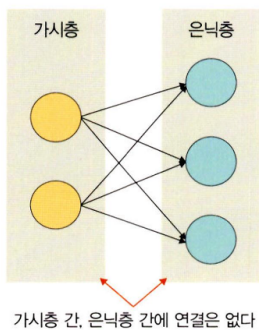
▼ 그림 4-26 순환 신경망



- 시계열 데이터(음악, 영상 등) 같은 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 인공 신경망
 - '순환'은 자기 자신을 참조한다는 것으로 → 현재 결과가 이전 결과와 연관이 있다는 의미

제한된 볼츠만 머신 (RBM)

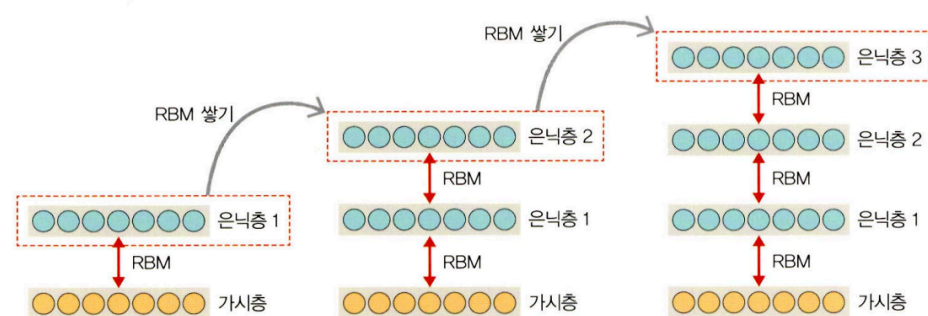
▼ 그림 4-27 제한된 볼츠만 머신



- 가시층(visible layer)과 은닉층(hidden layer)으로 구성된 모델
 - 가시층은 은닉층과만 연결 (가시층 간, 은닉층 간 연결은 없음)
- 차원 감소, 분류, 선형 회귀 분석, 협업 필터링(collaborative filtering), 특성 값 학습(feature learning), 주제 모델링(topic modeling)에 사용
- 기울기 소멸 문제를 해결하기 위해 사전 학습 용도로 활용 가능
- 심층 신뢰 신경망(DBN)의 요소로 활용

심층 신뢰 신경망 (DBN)

▼ 그림 4-28 심층 신뢰 신경망



- 입력층과 은닉층으로 구성된 제한된 볼츠만 머신을 블록처럼 여러 층으로 쌓은 형태로 연결된 신경망
 - 사전 훈련된 제한된 볼츠만 머신을 층층이 쌓아 올린 구조 → 레이블이 없는 데이터에 대한 비지도 학습이 가능
 - 부분적인 이미지에서 전체를 연상하는 일반화와 추상화 과정을 구현할 때 사용하면 유용
- 순차적으로 심층 신뢰 신경망을 학습시켜 가면서 계층적 구조 생성
- 비지도 학습으로 학습
- 위로 올라갈수록 추상적 특성을 추출

- 학습된 가중치를 다층 퍼셉트론의 가중치 초기값으로 사용
-